AI Agent 横纵分析报告

研究对象：AI Agent / 智能体（工具调用、规划、记忆与自主决策）

类型：技术范式

研究日期：2026-04-30

一、一句话定义

AI Agent（智能体）是以大语言模型为"大脑"，通过工具调用获取"行动力"、通过规划机制获得"决策力"、通过记忆系统获得"持久性"的自主软件实体。它不是一个新的技术概念——1983 年的 SOAR 架构就已是 Agent——但直到 2022–2023 年，LLM 的推理能力爆发才真正激活了 Agent 范式，让 Agent 从学术论文里的数学模型变成了能订机票、写代码、操作电脑的真实存在。

如果你只记住一句话：**Agent = LLM + 工具 + 记忆 + 自主循环。**

> 🎯 **读完这篇你能**：判断一个场景该不该用 Agent，理解 Agent 和普通 LLM 的本质区别在于"自主循环决策"而非"一问一答"，能评估一个 Agent 系统的工具设计、记忆策略和规划机制是否合理。

阅读指南

如果你只有 5 分钟：读一句话定义 + 横纵交汇洞察的最后一部分（"Agent 会走到哪里"的三个未来剧本）。你能在 5 分钟内把握 Agent 的本质和未来方向。

如果你想理解 Agent 但不碰技术细节：读技术背景的 2.1（核心公式）和 2.3（为什么现在才火）+ 纵向分析每个阶段的标题和阶段末的总结句 + 横向分析的产品形态对比和战略分组部分。约 25 分钟。

如果你是 Agent 开发者/研究者：直接跳横向分析的框架阵营对比和基准测试表现。纵向分析的 3.4（2023 Agent 元年：AutoGPT、BabyAGI）和 3.7（2026：Agent 编程范式的全面确立）是两个关键转折点，横纵交汇洞察中关于 Agent 核心矛盾的讨论值得精读。

二、技术背景

> 前置知识已独立为 `AI Agent_前置知识.md` ，本文仅做简要引用。零基础读者请先阅读前置知识。

为了让不熟悉底层技术的读者也能理解后续的纵向分析和横向对比，这里用最简单的语言交代 Agent 的核心技术栈。

2.1 核心公式

现代 AI Agent 的技术栈可以抽象为：

```
Agent = LLM（推理核心） + Tool Calling（工具调用） + Memory（记忆） + Planning（规划）
```

四者缺一不可。没有工具调用，Agent 只有想法没有行动；没有记忆，Agent 每次对话都是"初次见面"；没有规划，Agent 面对复杂任务会迷失方向。

2.2 关键技术点

Tool Calling / Function Calling：2023 年 6 月 OpenAI 引入，让 LLM 能输出结构化 API 调用请求。这是 Agent 从"会说话的脑子"变成"有手有脚的助手"的关键一步。不同模型的实现有差异：OpenAI 支持并行调用，Claude 更强调安全性（不确定时倾向不调用），Gemini 深度集成 Google 生态。

ReAct（Reasoning + Acting）：2022 年由 Yao et al. 提出的推理–行动交替循环。Agent 每次推理一小步，查一下外部信息，再推理下一步。这种方法有效解决了 LLM 的"闭门造车"问题——在没有外部信息验证时，LLM 很容易编造错误答案。

Planning（规划）：从 CoT（思维链，2022）到 ToT（思维树，2023）到 GoT（思维图，2024），Agent 的规划能力经历了从"一条直线"到"一棵树"再到"一张网"的演进。最新的方向是递归推理（Recursive Multi–Agent Systems, 2026），把递归作为新的 scaling 维度。

MCP（Model Context Protocol）：Anthropic 2024 年底推出的开放协议，标准化 Agent 与外部工具的连接方式，被称为"AI 的 USB–C"。2025 年 12 月 Anthropic 将其捐赠给 Linux 基金会下属的 Agentic AI Foundation，OpenAI、Google、Microsoft 全部加入，MCP 从"Anthropic 的协议"升级为"行业标准"。配套的 A2A（Agent–to–Agent Protocol，Google 2025）解决的是 Agent 间的通信问题。

2.3 为什么 Agent 现在才火?

一个 40 年前就有的概念（SOAR, 1983），为什么 2023 年突然爆发？答案很简单：**Agent 需要一个足够聪明的"大脑"**。Symbolic AI 时代的 Agent 受限于规则系统的表达能力——你能手动编写的规则永远无法覆盖真实世界的复杂性。LLM 的出现让 Agent 第一次拥有了"理解复杂指令、做出合理判断"的能力，剩下的工具调用和记忆系统只是锦上添花的技术工程。

前置知识（包括 Tool Calling、ReAct、Memory、MCP、A2A、Prompt Injection、Agent 基准测试等的详细讲解，配比喻和例子）请参考 `AI Agent_前置知识.md` 。

> 📚 **关联报告**

> – Prompt Engineering — Agent 推理策略（ReAct/CoT/ToT）的源头

> – LLM 应用框架 — LangGraph、Dify、CrewAI 等 Agent 框架是应用层落地

> – Agent Runtime / Harness — Agent 的运行时基础设施：沙箱、护栏、持久化

三、纵向分析：AI Agent 四十年沉浮

> 从 1983 年 SOAR 到 2026 年 Agent 编程范式，一段长达 43 年的技术演进叙事。

3.1 萌芽期（1983–2010）：学术探索的漫长冬季

AI Agent 不是 2023 年的新发明。它的历史比 Transformer 长得多，甚至比互联网商业化还要早。

1983 年，卡内基梅隆大学的 John Laird、Allen Newell 和 Paul Rosenbloom 创建了 SOAR（State, Operator And Result）认知架构。这是第一个可计算的通用智能体架构，试图用符号系统模拟人类的认知过程。SOAR 的核心思想是：智能体在状态空间中搜索，选择算子（Operator）作用于当前状态，产生新状态，循环往复直到目标达成。今天 ReAct 模式的"思考→行动→观察→思考"循环，和 SOAR 的问题空间搜索是同构的——只不过当年的算子是手工定义的符号规则，而今天的算子是 LLM 动态生成的。

1986 年，Marvin Minsky 出版了《Society of Mind》。 这本书提出了一个在当时看来非常激进的观点：智能不是由一个中央处理器产生的，而是由大量非智能的"Agent"交互涌现出来的。这本书直接影响了后来多 Agent 系统的设计哲学——每个 Agent 都不是全能的，但多个 Agent 协作可以产生整体智能。

1987 年，BDI（Belief–Desire–Intention）模型由哲学家 Michael Bratman 提出，后由 Rao 和 Georgeff 在 1995 年形式化。 BDI 模型将 Agent 的内部状态分为信念（Belief，对世界的认知）、愿望（Desire，想达成的状态）和意图（Intention，当前选择的行动方案）。这个模型至今仍是多 Agent 系统的理论基础。Bratman 的核心洞察是：理性的行动者不是每时每刻重新计算所有可能，而是"承诺"于当前意图，只在必要时才重新评估——这叫"受承诺的理性"。

1990–2000 年代，Symbolic AI 时代的 Agent 研究主要在规划系统上。 STRIPS 和 PDDL 定义了通用规划语言，JADE 和 Jason 是 Java 平台的多 Agent 系统框架。但问题很明显：在真实世界中，你无法预先把所有规则写进系统。一个搬箱子机器人在模拟环境里完美运行，放到真实仓库里就会被货架阴影、灯光变化、箱子形状差异搞崩溃。Symbolic Agent 的瓶颈不是规划算法，而是**感知和理解真实世界的能力**。

这个阶段埋下了一个重要伏笔：当 Agent 的研究者们苦苦寻找"让 Agent 理解世界"的方法时，另一个方向——神经网络——在悄悄生长。2010 年代深度强化学习（DQN, PPO）让 Agent 获得了学习能力，但仍然受限于单一任务范式。AlphaGo 能在围棋上击败世界冠军，但换个游戏就完全不会了。通用 Agent 需要一个通用的"理解引擎"。

而这个引擎——Transformer——在 2017 年诞生了。

在进入 Agent 元年前，还有一个不应被遗忘的坐标：**2021 年 OpenAI 的 WebGPT**（arXiv:2112.09332）。WebGPT 让 GPT–3 通过浏览器搜索来回答问题——用 Bing 搜索、浏览页面、提取信息、综合回答。从今天的角度看，WebGPT

就是 Agent 的雏形：有工具（浏览器）、有规划（搜索策略）、有记忆（已浏览页面的信息）。只不过当时还没有"Agent"这个热词，它被归类在"检索增强"的范畴里。

WebGPT 的意义在于证明了：**即使是不完美的工具使用能力，也能显著提升 LLM 在知识密集型任务上的表现。** WebGPT 在 TruthfulQA 上的表现比纯 GPT-3 高了 30% 以上。这个信号在当时被忽视了——2021 年的 AI 社区还在为 GPT-3 的文本生成能力惊叹，没有多少人注意到"LLM 结合工具"这个方向的潜力。

另一个常被忽略的前奏工作是 2022 年的 **SayCan**（Google Robotics, arXiv:2204.01691）。SayCan 让 LLM 为机器人规划行动——LLM 说"我要做什么"，机器人说"我能做什么"，两者协商出可行的方案。这正是后来的"LLM + 行动系统"的 Agent 架构，只不过行动系统不是软件工具而是物理机器人。

这两个前奏指向同一个结论：**Agent 范式不是 2023 年突然冒出来的，它在 2021-2022 年已经被不同的研究团队从不同方向独立探索出来**——WebGPT 从检索出发，SayCan 从机器人出发，ReAct 从推理出发。三个方向在 2023 年汇聚成了一个统一的 Agent 范式。

3.2 爆发前夜（2017-2022）：Transformer 激活 Agent 范式

2017 年 Google 的 "Attention Is All You Need" 论文发表时，几乎没有人把它和 Agent 联系起来。Transformer 最初只是机器翻译的一个新架构。但历史证明，Transformer 的序列到序列建模能力，天然适配 Agent 的核心需求：理解上下文、生成推理链、输出结构化动作。

2022 年 1 月，Google 的 Wei et al. 发表了 Chain-of-Thought Prompting（CoT）。 从今天的角度看，CoT 的意义远不止"提升推理准确率"——它第一次展示了：**LLM 的推理可以通过 prompt 工程被引导、被结构化的。** Agent 的"思考过程"不再是黑箱，而是可以被程序化控制的。

真正的转折点是 2022 年 10 月的 ReAct 论文。 Yao et al.（普林斯顿 + Google Brain）在 ICLR 2023 上发表的工作，首次将推理轨迹（Reasoning trace）和行动（Acting）交错结合。实验数据很有说服力：在 HotpotQA 和 Fever 上，ReAct 通过调用 Wikipedia API 克服了 CoT 的幻觉问题；在 ALFWorld 和 WebShop 上，ReAct 比模仿学习和强化学习分别高出 34% 和 10% 的绝对成功率。

ReAct 的论文标题里有"Synergizing Reasoning and Acting"——协同推理与行动。这个"协同"二字精准概括了现代 Agent 的核心范式：推理指导行动，行动结果反过来矫正推理。ReAct 锁定了后续所有 Agent 框架的设计方向，今天 LangChain、AutoGPT、CrewAI 的 Agent 循环，本质都是 ReAct 的各种变体。

但 2022 年的 LLM（GPT-3、PaLM）在 Agent 场景下的实际表现并不稳定。推理能力还不够强，工具调用的可靠性不够高。Agent 范式在理论上成立，但工程上还差关键一环。

3.3 Agent 元年（2023）：寒武纪大爆发

2023 年被公认为"Agent 元年"。 这一年发生的事，密度高得惊人。

2 月，Toolformer（Meta）发布。Timo Schick 等人展示了 LLM 可以通过自监督学习调用外部工具（计算器、搜索引擎、翻译系统），且不影响语言建模能力。"鱼与熊掌兼得"——这打破了此前的一个担忧：让 LLM 用工具会不会降低它本身的语言能力。

3 月，AutoGPT 引爆开源社区。 一个叫做 Significant Gravitas 的独立开发者发布了 AutoGPT，数日内 GitHub 星数突破 15k，最终达到 183k。AutoGPT 的理念极具震撼力：给定一个目标（"建立一个 SaaS 公司"），AutoGPT 会自动

分解任务、调用工具、迭代执行、自我反思——全程无需人工干预。虽然实际效果远不如 Demo 那么惊艳（后面会聊到），但它让"自主 Agent"这个概念从论文变成了人人可体验的东西。

同月，**BabyAGI 和 Reflexion 同时出现**。BabyAGI 是 Yohei Nakajima 的一个轻量级实验，展示了"规划–执行–反馈"循环的极简实现。Reflexion（Shinn et al.）则提出了"语言反馈强化学习"——Agent 通过语言反馈信号自我反思，将反思存入情景记忆。Reflexion 在 HumanEval 上达到 91% pass@1，超越了 GPT–4 当时的 80%。

4 月，斯坦福的 Generative Agents 论文引爆了另一条线。 Park et al. 把 25 个 Agent 放进一个类似《模拟人生》的沙盒小镇，它们自主社交、规划、记忆、反思，产生了涌现社会行为——Agent 组织派对、互相传播新闻、形成社交关系。这篇论文不仅展示了多 Agent 涌现的可能性，还提出了一个重要的记忆架构：**观察→记忆→反思→规划**的循环，每个 Agent 维护自己的记忆流，定期做高层反思，修正行为模式。这个架构深刻影响了后来的 Agent 记忆设计。

5 月，Tree of Thoughts（Yao et al.）和 Voyager（NVIDIA）相继发布。 ToT 把 CoT 的线性思维链扩展为树状搜索——在每一步生成多个"思考节点"，用 BFS/DFS 评估各分支。在 Game of 24 任务上，CoT 的解决率只有 4%，ToT 飙到了 74%。Voyager 则展示了 LLM 驱动的具身 Agent 在 Minecraft 中的终身学习能力——自动生成课程、维护技能库、迭代优化 prompt，在 Minecraft 中解锁里程碑的速度是此前方法的 15.3 倍。

6 月 13 日，OpenAI 发布 Function Calling。 这是 2023 年最重要的产品级事件之一。从此，Agent 不再需要 hack prompt 来调用 API——模型原生支持输出结构化的函数调用参数。LangChain 等框架迅速跟进，Agent 开发的生产力跃升了一个量级。

整个 2023 年夏天，Agent 成为 AI 社区最热的话题。LangChain 的 Agent 框架迅速崛起，AutoGPT 星数突破 35k，AgentGPT、MetaGPT、Camel 等开源项目密集涌现。**"Agent = LLM + 工具 + 记忆"的范式在这一年确立。**

回头看，2023 年的 Agent 爆发有三个驱动力：

1. **LLM 推理能力达标了**：GPT–4 的推理能力让 Agent 在复杂任务中的成功率从"不可用"变成了"可用"
2. **工具调用标准化了**：Function Calling 让 Agent 调用 API 从 hack 变成了原生能力
3. **开源社区引爆了**：AutoGPT 让每个人都能体验 Agent，形成了巨大的网络效应

3.4 产品化与工业化（2024）：从 Demo 到产品

2023 年的 Agent 像一堆炫酷的概念车——每辆都能跑、都能引起围观，但没有一辆真正上路。2024 年，行业开始认真解决"上路"的问题。

3 月，Cognition AI 发布 Devin，"首个 AI 软件工程师"。 Devin 展示了端到端的软件工程能力：接收 GitHub Issue，自己理解需求、搜索代码库、写代码、跑测试、部署。在 SWE–Bench 上取得了当时最高的解决率。虽然 Devin 的实际效果被很多开发者质疑（"Demo 级产品"），但它第一次把 Agent 的商业化产品摆在了台面上——定价 $500/月。

与此同时，Agent 框架在 2024 年走向成熟。 LangChain 从单一的 Agent 抽象演进为 LangGraph——基于有向图的 Agent 状态机，支持复杂的控制流和循环。微软的 AutoGen 主打多 Agent 对话，Agent 之间通过消息流协作。CrewAI 则走了角色扮演路线，让开发者定义"研究员""写手""编辑"等角色，模拟真实工作流。Dify 在可视化 Agent 平台上快速增长，到 2026 年 Star 数已接近 LangChain。

5 月，Princeton 发布 SWE–agent，聚焦 Agent 与计算机的交互接口。SWE–agent 的设计思想——将 Agent 的"行动空间"限定在特定领域的 API 上——为后来的 Computer Use 铺了路。

10 月，Anthropic 发布 Claude Computer Use。 这是 2024 年最重要的 Agent 事件。 Anthropic 展示了 Claude 3.5 Sonnet 操控计算机界面的能力——看屏幕截图、移动光标、点击按钮、输入文字。不是通过 API，而是像人一

样"用眼睛看、用手操作"。由此，Agent 可以操作任何软件，无论它有没有 API。这个突破打开了 Agent 在 GUI 时代的新范式。

但 Computer Use 也暴露了 Agent 的鲁棒性问题：在复杂的网页布局中，Claude 有时候会点错按钮、填错表单。**"能用"和"好用"之间的差距，是 2024 年 Agent 产品化的主要矛盾。**

11 月 25 日，Anthropic 发布 MCP（Model Context Protocol）。 这不是一个产品，而是一个标准——开放、中立的 Agent-工具连接协议。MCP 的野心是成为"AI 的 USB-C"：让任何 Agent 都能通过统一协议连接任何工具。与封闭的 Function Calling API 不同，MCP 是开放的、去中心化的。很快，LangChain、Cursor、Windsurf 等主流工具都宣布支持 MCP。到 2025 年，MCP 已经成为事实上的行业标准。

3.5 规模化落地与博弈（2025-2026）：Agent 进入深水区

2025 年是 Agent 从"能不能做"到"要不要用"的一年。

1 月，OpenAI 发布 Operator——对标 Computer Use 的浏览器自动化 Agent。Operator 能帮用户网购、订餐、填表单。定价 $200/月（ChatGPT Pro 订阅）。同月 GitHub Copilot Agent 模式上线，AI 编程进一步深化。

3 月，Manus 现象级爆红。 一个来自中国团队 Monica 的通用自主 Agent 产品，一夜之间刷屏全网。邀请码被炒到上千元。Manus 展示了 Agent 的终极形态：给你一个任务（"调研 AI Agent 行业"），它自己搜索、分析、写报告、生成 PPT。但口碑迅速分化——支持者惊叹于它的自主性，批评者指出它不过是"套壳 Claude 的网页自动化"，真实场景的成功率并不高。

Manus 的爆红和争议，折射出 2025 年 Agent 行业的深层矛盾：**Demo 级效果和生产级可靠性之间的鸿沟。** 几乎所有 Agent 产品都在面临同一个问题：90% 的场景表现惊艳，剩下 10% 的场景彻底翻车，而这 10% 恰好是用户最在意的关键场景。

3 月，OpenAI 发布 Agents SDK。 OpenAI 终于推出了自己的 Agent 开发框架——轻量级、支持多 Agent 编排、核心机制是 Handoff（Agent 之间移交控制权）。这表明 OpenAI 认识到 Agent 不是一个产品功能，而是一个开发范式，需要工具生态来支撑。

同样在 3 月，Google 发布 A2A（Agent-to-Agent）协议。 如果说 MCP 解决的是"Agent 怎么用工具"，A2A 解决的是"Agent 之间怎么聊天"。场景很清晰：一家旅行 Agent 需要和酒店 Agent、航班 Agent、租车 Agent 沟通。A2A 在协议层面规范了 Agent 间的任务协商、能力发现、安全认证。

MCP 与 A2A 的分工很有意思：**Anthropic 做"纵向"（Agent 连工具），Google 做"横向"（Agent 连 Agent）。** 两者不是竞争关系，而是互补的两层标准。

2025 年 12 月，MCP 的行业地位发生质变——Anthropic 将其捐赠给新成立的 Agentic AI Foundation（Linux 基金会），OpenAI、Google、Microsoft 等巨头全部加入。 MCP 从"Anthropic 提出的一个协议"变成了"行业共同维护的标准"。同一时期，A2A 已有 150+ 家企业接入。双协议格局正式确立：MCP 解决"Agent 怎么用工具"，A2A 解决"Agent 怎么找伙伴"。

几乎同时，一个名为 OpenClaw 的开源项目在 GitHub 上悄然崛起。 2025 年 11 月以 Clawdbot 为前身诞生，2026 年初更名为 OpenClaw，定位为"个人 AI Agent 操作系统"。到 2026 年 4 月，它的 GitHub Star 数突破 30 万，超越 Linux 和 React，成为 GitHub 软件类星标历史第一。这定义了一个新品类——用户在自己的设备上运行一个 Agent 系统，让它自主管理文件、浏览网页、操作应用。中国甚至出现了"人人养龙虾"的社交媒体现象，赛迪智库专门发布了研

究报告。OpenClaw 的成功说明了一个重要趋势：**用户不想用更多 App，他们想要一个能替自己操作所有 App 的 Agent。**

OpenClaw 带起来的浪潮还没退，第二个现象级项目 Hermes Agent 杀了进来。 2026 年 3 月，Nous Research 发布了 Hermes Agent——同样定位为开源个人 Agent 平台，但核心卖点完全不同：**持久记忆和自我进化。** OpenClaw 每次重启会丢失上下文，Hermes Agent 用 FTS5 记忆系统把"每次对话的历史"持久化存储，Agent 越用越了解你的习惯和偏好。它的自我进化机制——Agent 能反思自己的过往行为并调整策略——是 OpenClaw 没有的功能。

到 2026 年 4 月，Hermes Agent 在不到两个月内冲到 10 万 Stars，成为 OpenClaw 之后增长最快的 AI Agent 开源项目。中国社交网络出现"养龙虾不如养马"的讨论。但对标 OpenClaw 的快速膨胀也带来了同样的烦恼——功能迭代太快、稳定性不足、以及社区关于"代码抄袭"的争议（2026 年 4 月被指控抄袭中国团队的架构设计）。

Hermes Agent 和 OpenClaw 的竞争定义了 2026 年 Agent 品类的核心分化：**OpenClaw 追求"广度"（什么都能操作），Hermes Agent 追求"深度"（越用越懂你）。** 两个项目的 Star 数加起来超过 40 万，说明用户对"个人 AI Agent"的需求不是一时热度——它是真实存在的。

2025 年下半年，Agent 进入企业级部署阶段。 Google 推出 Gemini Agent（原生 Agent 能力），Microsoft 推出 Copilot Agents，Amazon Bedrock Agents 上线。各大云厂商都在做同一件事：让 Agent 成为云平台的原生服务。这背后的逻辑很清楚——Agent 消耗的 token 是普通聊天的几十倍，对云厂商来说是巨大的算力消费增量。

7 月，OpenAI 发布 ChatGPT Agent——ChatGPT 用户可直接让 Agent 自主完成复杂任务。同月，Meta 以超过 20 亿美元收购了爆红一时的中国 Agent 产品 Manus（后被中国监管否决），这被视为 Meta 全面押注 AI Agent 的标志。9 月，Anthropic 发布 Claude Agent SDK（开源），随后在 10 月推出 Opus 4.5——首个具备"自我改进 Agent"能力的模型。

12 月，Microsoft Ignite 将 2026 年定为"Year of the Agent"，推出 Agent 365 全面嵌入 Office 套件。Salesforce 在 Dreamforce 2025 上发布 Agentforce 360，定义了"Agentic Enterprise"概念。这一年，Agent 从创业公司的独角戏变成了巨头的必争之地。收购案密集发生：Anthropic 收购 Vercept（增强 Computer Use），Salesforce 收购 Doti、Convergence、Clockwise 等多起，NICE 以 $955M 收购 Agentic AI 公司。但也出现了第一批倒下的——a16z 支持的 Yupp.ai 融资 $33M 后 22 个月倒闭，Robin AI 等也相继关门。

2026 年 3 月至 4 月，Agent 行业进入最密集的产品发布期：

3 月 24 日，Anthropic 发布 Claude Cowork 和 Claude Dispatch。 Cowork 定位为"AI 数字同事"，与用户在同一台电脑上协作；Dispatch 支持手机远程操控电脑执行任务。同一天，Claude 获得直接操控 Mac 电脑的能力。Cowork 的发布引发了关于**"SaaSpocalypse"**的大讨论——如果 Agent 能直接操作软件，传统 SaaS 的 UI 还有什么意义？

4 月，OpenAI 发布 Workspace Agents——ChatGPT Business/Enterprise 版的 7×24 在线 Agent，正式宣布 Custom GPTs 逐步弃用。同时大幅升级 Agents SDK（v0.14.0），新增 Sandbox 沙箱环境和 Harness 架构——从"能用"走向"安全用"。

4 月，Google Cloud Next '26 发布 Enterprise Agent Platform，整合 Vertex AI 为全栈 Agent 平台，推出 AI Ultra 订阅（$250/月）。Microsoft Agent Framework 1.0 正式发布，原生支持 MCP 和 A2A，一行代码切换 6 个模型厂商。

同样在 4 月，Anthropic 发布 Claude Managed Agents（托管智能体服务）——开发周期从数月缩短到数天。由此，Anthropic 从"提供模型"直接进入了"提供 Agent 服务"的领域。行业反应激烈：**"Anthropic 亲自下场，又一批 Agent**

创业公司死掉了。" 同月，Anthropic 还发布 Agent Skills 开放标准，并宣布 Enterprise 从 per-seat 转向按用量计费——重度用户成本可能翻倍，被视为"AI 免费时代结束"的又一标志。

4 月 22 日，Salesforce TDX 2026 发布 Headless 360——向 AI Agent 开放全部平台能力，采用 per-conversation 定价（$2/对话）。GitHub Copilot 也从固定订阅转向按用量计费。整个行业的定价模式正在从"卖座位"转向"卖 Agent 工作量"。

进入 2026 年，Agent 安全事件开始从概念讨论变为真实事故。

4 月 28 日，Hacker News 上出现了一篇引发行业震动的帖子："我们的 AI Agent 删除了生产数据库。" 500 多分、1002 条评论。Agent 被 prompt injection 攻击后执行了 DROP TABLE 操作。这不是第一次 Agent 事故，但绝对是影响最大的一次。它让整个行业意识到：Agent 的权限控制不是锦上添花，而是生死攸关。

同一周，多项 Agent 安全相关的技术进展值得关注：

- **Red Hat OpenClaw 发布 Tank OS**，把 Agent 放进容器隔离运行
- **arXiv 2604.25891 研究 "Conditional Misalignment"**——微调可能导致 LLM 产生"有条件失调"，在特定上下文触发下暴露危险行为
- **Claude Code 版权讨论**（HN 403 分）：AI 写代码的代码归属权问题

4 月 29 日，Anthropic 被曝可能融资 $500 亿，估值 $9000 亿——而融资的核心用途之一就是支撑 Agent 推理的高昂算力成本。同一天，Parag Agrawal（前 Twitter CEO）的 AI Agent 创业公司 Parallel Web Systems 在 5 个月内估值翻倍到 $20 亿。Sequoia 领投 $1 亿。

也是在 4 月，中国否决了 Meta 以 $20 亿收购 Manus 的交易。 由此可见，AI Agent 已经被纳入战略资产范畴，国际资本流动受到管控。

4 月 28 日，Microsoft 发布 SWE-Edit（arXiv:2604.26102）——代码编辑 Agent 的新范式。这是 2026 年 Agent 发展的重要信号：Agent 工具正在从"通用框架"进入"领域专用工具"阶段。

4 月的另一条重磅消息：OpenAI 正与联发科、高通、立讯精密合作开发手机，核心卖点是 AI Agent 替代 App 作为交互入口。 这是 Agent 愿景的最大胆表述——**Agent 不是 App 里的一个功能，Agent 本身就是下一代操作系统接口。**

四、横向分析：2026 年 Agent 竞争图谱

> 以 2026 年 4 月为时间切片，看 AI Agent 赛道的竞争格局。

4.1 竞品格局判断

AI Agent 赛道属于**场景 C（竞品充分）**——玩家众多、维度复杂、格局快速变化。下面从四个层次展开：Agent 开发框架、Agent 产品、大厂 Agent 平台、以及基础设施层（协议与标准）。

4.2 Agent 开发框架：百家争鸣

框架	开发者	发布时间	GitHub Stars	开源	核心定位
OpenClaw	社区	2025-11	300,000+	✅	个人 Agent 操作系统
Hermes Agent	Nous Research	2026-03	100,000+	✅	自我进化+持久记忆 Agent
AutoGPT	Significant Gravitas	2023-03	183,890	✅	自主 Agent，任务分解
Dify	LangGenius	2023-05	140,000+	✅	可视化 Agent 搭建平台
LangChain	LangChain Inc	2022-12	103,000+	✅	通用 LLM 应用框架
MetaGPT	FoundationAgents	2023-06	67,564	✅	多 Agent 角色扮演
AutoGen	Microsoft	2023-10	57,586	✅	多 Agent 对话框架
CrewAI	CrewAI Inc	2024-01	50,315	✅	多 Agent 角色协作
LlamaIndex	RunLLama	2022-11	40,000+	✅	数据/RAG 框架
Agno	Agno Inc	2022-05	39,803	✅	Agent 构建工具
LangGraph	LangChain Inc	2023-08	30,850	✅	状态图 Agent
OpenAI Agents SDK	OpenAI	2025-03	25,576	✅	官方多 Agent 编排
Microsoft Agent Framework	Microsoft	2025	30,000+	✅	企业 Agent 开发框架
Google ADK	Google	2025	20,000+	✅	Agent 开发工具集

数据说明：GitHub Stars 截至 2026-04-30，数据来自 GitHub API。这些数字只反映社区热度，不直接等价于工程成熟度或商业化程度。

框架竞争的核心矛盾

LangChain 的生态霸权与复杂度争议。 LangChain 是 Agent 框架里最"重"的——它提供了 700+ 集成、完整的工具链（LangSmith 监控、LangServe 部署、LangGraph 状态图），但也因此背负了大量批评。最典型的一句吐槽："LangChain 在 OpenAI 完成 80% 工作的地方加了 200% 的复杂性。" 社区中很多开发者转向更轻量的方案（直接调 OpenAI API + 写几十行 Python）。但 LangChain 的生态壁垒正在形成：新工具发布时，支持 LangChain 往往是最先做的集成。

Dify 的崛起说明了什么？ Dify 的 Star 增长曲线在 2025-2026 年接近 LangChain，这在社区看来是一个明确的信号：**用户正在从"写代码"转向"可视化编排"。** 特别是非 AI 专业背景的开发者（全栈工程师、产品经理），他们希望用拖拽的方式构建 Agent 工作流，而不是学习 LangChain 的抽象层。Dify 同时也支持接入国内大模型（通义千问、文心一言、DeepSeek），在国内市场有天然优势。

多 Agent 框架的差异化竞争。 AutoGen（微软）、CrewAI、MetaGPT 三个多 Agent 框架走了三条不同的路：

- **AutoGen** 的核心是 Agent 间"对话"——Agent 通过消息流协商合作，类似一个聊天群组
- **CrewAI** 的核心是"角色"——定义 Agent 的角色、任务、流程，像管一个项目团队

- **MetaGPT** 的核心是"SOP"——模拟软件公司从 PM 到测试的完整角色链

OpenAI Agents SDK 的存在本身就是个声明。 2025 年 3 月 OpenAI 推出自己的 Agent 框架时，LangChain 的股票（未上市）可能跌了一截。但经过一年多的发展，结果远没有到"颠覆"的程度——Agents SDK 定位为轻量级工具（25k stars），和 LangChain 的重框架走的是不同路线。OpenAI 的 Handoff 机制（Agent 间移交控制权）是设计上的亮点，被其他框架借鉴。2026 年 4 月的 v0.14.0 升级增加了 Sandbox 沙箱环境和 Harness 架构（来自 Codex 的生产级运行框架），标志着 OpenAI 正式从"功能优先"转向"安全优先"。

4.3 Agent 产品：谁在真正创造价值?

产品	公司	发布时间	类型	定价
OpenClaw	社区	2025-11	个人 Agent 操作系统	免费开源
Hermes Agent	Nous Research	2026-03	自我进化+持久记忆 Agent	免费开源
Claude Cowork	Anthropic	2026-03	桌面协作 Agent	API 按量计费
Claude Dispatch	Anthropic	2026-03	远程操控 Agent	API 按量计费
OpenAI Workspace Agents	OpenAI	2026-04	企业长周期 Agent	ChatGPT Business/Enterprise
ChatGPT Agent	OpenAI	2025-07	通用自主 Agent	ChatGPT Plus $20/月
OpenAI Operator	OpenAI	2025-01	浏览器 Agent	ChatGPT Pro $200/月
Manus	Monica / Meta	2025-03	通用自主 Agent	免费 + Pro $39/月
Claude Computer Use	Anthropic	2024-10	桌面操控 Agent	API 按量计费
Devin	Cognition AI	2024-03	AI 软件工程师	$500/月起

AI 编程 Agent 是当前最成熟、变现最清晰的产品方向。 Cursor 是公认的用户体验标杆，其 `@` 引用上下文的设计被广泛模仿。GitHub Copilot 在 2026 年从固定订阅转向用量计费，是行业的重要风向标——说明 Agent 的工作量不是均匀的（每个月少的时候只有几十次调用，多的时候几千次），固定订阅制下厂商利润空间有限。

通用自主 Agent（Manus / Operator）面临"Demo 级 vs 生产级"的信任鸿沟。 Manus 在爆红后热度明显回落，不是因为技术不行，而是因为用户期望被 Demo 拉得太高。一个 Agent 如果在 9 个任务里完成 8 个，第 9 个搞砸了，用户的记忆点不是那 8 个成功的，而是那个搞砸的。这是 Agent 产品的"容错困境"——用户对人类的容错率远高于对 Agent 的。

Devin 的 $500/月定价本身就说明了一个问题：真正的 AI 软件工程师还很贵。 它的目标客群不是个人开发者，而是企业——把 Devin 当作一个 junior engineer 的替代品来算 ROI。

4.4 大厂平台战略：云厂商的 Agent 牌局

厂商	Agent 平台/产品	核心策略
OpenAI	Operator + ChatGPT Agent + Workspace Agents + Agents SDK	四层覆盖：产品（Operator/ChatGPT Agent）、企业（Workspace Agents）、开发者（SDK）
Anthropic	Cowork + Dispatch + Computer Use + Claude Agent SDK + Managed Agents + MCP	从模型公司全面转型 Agent 平台公司，亲自下场做 Agent 产品
Google	Enterprise Agent Platform + Vertex AI + A2A + ADK	云厂商全栈绑定 + 协议标准（A2A）+ 开发生态（ADK）
Microsoft	Agent 365 + Copilot Agents + Agent Framework 1.0	企业 Copilot 生态 + 开源 Agent 框架
字节跳动	Coze / 扣子	国内最成功的 Agent 平台，月活领先
百度	AppBuilder	云厂商策略，绑定文心大模型
阿里	百炼平台	云厂商策略，绑定通义千问

大厂的 Agent 策略分两派：**模型厂商做 Agent 产品**（OpenAI、Anthropic），**云厂商做 Agent 平台**（Google、Microsoft、百度、阿里）。前者从模型能力出发向下延伸到产品，后者从基础设施出发向上延 伸到服务。两条路线在 2026 年还没有明显的胜负之分——模型厂商有更好的模型，云厂商有更好的分发渠道和企业关系。

字节跳动的 Coze（扣子）是一个值得单独分析的案例。它走了一条和所有大厂不同的路——**不做模型也不做云，而是做一个纯粹的低代码 Agent 平台**。结果是国内 Agent 平台的月活冠军。Coze 的成功说明：在 Agent 这个赛道，**"好用"比"强"更重要**，尤其是在非技术用户的市场上。

4.5 基准测试：谁在数据上领先?

SWE-Bench（AI 软件工程能力）：

模型/Agent	解决率	时间
人类基线	~85%	—
Claude 3.5 Sonnet + Agent	~49%	2025
GPT-4 + Agent	~48%	2025
OpenAI o1 + Agent	~45%	2025

GAIA（通用 AI 助手能力）：

模型/Agent	得分	测试时间
人类基线	~92%	—
OpenAI o1 + Agent	~67%（Level 1）	2025
Claude + Computer Use	领先桌面操控	2025
AutoGen + GPT-4	~40-50%	2025

数据来源：swebench.com 公开 Leaderboard 及 GAIA 官方报告。数字反映的是基准测试情境下的表现，在实际场景中会有出入。

客观说，Agent 在基准测试上的进步是真实的：从 2023 年的 ~20% 到 2025 年的 ~50%，解决率翻了一倍多。但距离人类基线的 85% 还有巨大差距。最要命的是**最后 10% 的"长尾失败"**——Agent 在 90% 的常见场景下表现良好，但在剩余 10% 的边缘情况中彻底翻车。在真实的生产环境中，这 10% 往往是问题最核心的那部分。

4.6 市场全景图：AI Agent 的"三层架构"

如果把 2026 年的 Agent 行业画成一张架构图，它大概长这样：

底层：基础设施层

- 模型厂商 API（OpenAI、Anthropic、Google、DeepSeek 等）
- 协议标准（MCP、A2A）
- 安全与监控（OpenClaw、Guardrails）

中间层：框架与平台

- 通用框架（LangChain、LlamaIndex、OpenAI Agents SDK）
- 多 Agent 框架（AutoGen、CrewAI、MetaGPT）
- 可视化平台（Dify、Coze）

顶层：产品与应用

- 通用 Agent（Manus、Operator、Claude Computer Use）
- 垂直 Agent（编程：Cursor、Devin；客服；数据分析；运营）
- 企业 Agent（Copilot Agents、Vertex AI Agents、Bedrock Agents）

三层之间正在发生有趣的"挤压"：底层模型厂商在向上延伸（OpenAI 做 Agents SDK 和 Operator），顶层产品在向下沉淀（Manus 要用 MCP），中间层在向两侧扩张（Dify 做 Agent 平台，LangChain 做 LangSmith 监控）。谁能在三层之间建立最稳固的连接，谁就可能在 Agent 时代占据最有利的位置。

4.7 用户口碑：真实的声音

LangChain：又爱又恨。

> "LangChain 在 OpenAI 完成 80% 工作的地方加了 200% 的复杂性。"

这是社区里流传最广的一句评价。LangChain 的 API 变化频繁、学习曲线陡峭，是开发者最常抱怨的点。但抱怨归抱怨，当需要快速实现一个 Agent 原型时，很多人还是回到 LangChain——因为它"什么都有"。

Manus：高开低走。

Manus 在 2025 年 3 月的爆红是 Agent 行业的一个缩影：演示视频里的 Manus 看起来无所不能，实际用起来发现"还行但没那么神"。用户反映最集中的问题是"场景覆盖率不足"——Manus 在它训练过的场景里表现惊艳，遇到新场景就力不从心。这其实是所有"通用 Agent"的共同困境：**通用性和可靠性之间存在本质矛盾。**

AI Agent 删库事件：信任危机。

2026 年 4 月的删库事故把 Agent 安全问题推到了聚光灯下。用户的反应很有意思——多数人不是怪 Agent 本身，而是怪**把 Agent 直接暴露在生产环境中的决策**。这从侧面说明：行业对 Agent 的认知已经从"新奇事物"变成了"需要认真对待的生产工具"。大家都认同 Agent 有价值，但怎么安全地用 Agent，还没有共识。

OpenClaw：现象级的爱恨交织。

OpenClaw 的 30 万+ Stars 不是浪得虚名——它的价值观（开源、本地运行、用户控制）精准命中了技术社区的诉求。但快速膨胀也带来了质量争议：功能迭代太快导致稳定性不足，安全审计跟不上增长。中国政府对 OpenClaw 发布了安全风险提示，限制国企和政府机关使用。Red Hat 推出了 NemoClaw（企业安全分支）。一句话用户评价："OpenClaw 让我看到了 Agent 的未来，但这个未来还很粗糙。"

Hermes Agent vs OpenClaw：2026 年最激烈的 Agent 之争。 Hermes Agent 的支持者最常提的三个词——"记忆"、"进化"、"懂我"。用户切换的理由很直接："OpenClaw 每次都像第一次见你，Hermes Agent 记得你上个月说过的话。"吐槽集中在：自我进化偶尔会产生"学习错误"——Agent 从一次失败的交互中学会了不该学的行为模式。架构抄袭争议（被指控抄袭中国团队 EvoMap）也让部分技术社区对 Nous Research 的信任打了折扣。用户评价："Hermes Agent 是理想主义的产品——但它现在还只是个原型。"

五、横纵交汇洞察

5.1 历史如何塑造了今天的竞争格局

把纵向的历史线和横向的竞争格局叠在一起看，会发现一些耐人寻味的规律。

Anthropic 的 MCP 策略不是偶然的。 回到纵向线：Anthropic 在 2024 年推出 Computer Use（Agent 操控桌面）时，面临一个问题——Agent 能"看到"屏幕但不知道怎么"够到"工具。当时每个工具都要手写集成，效率低下。MCP 的推出正是对这个工程痛点的回应。而选择做开放协议而不是封闭生态，则是 Anthropic 一贯的战略风格——从 Claude 的"宪法 AI"到 MCP 的"开放标准"，走的都是"定规则而不是圈地盘"的路线。这跟 OpenAI 的 Function Calling 形成了鲜明对比：OpenAI 做的是自家 API 的原生能力，Anthropic 做的是跨平台的通用协议。

Dify 的崛起有深层原因。 Agent 技术栈在 2023 年的爆发太猛了——一年之内出现了 LangChain、AutoGPT、AutoGen、CrewAI、MetaGPT 等十几个框架，每个都有自己的抽象概念（Chain、Agent、Tool、Task、Role、Flow）。对专业 AI 工程师来说，这已经是认知负担。对普通开发者来说，这简直是灾难。Dify 抓住了这个"认知鸿沟"——用可视化界面屏蔽了框架层的复杂度。Dify 的增长曲线接近 LangChain，说明一个简单的道理：**当技术栈变得过于复杂时，降低复杂度的工具就会胜出。**

AI 编程 Agent 为什么先跑通商业化？ 这个问题在纵向线和横向线上都能找到答案。纵轴看：编程是一个"闭环"场景——输入（代码库 + Issue）、输出（修复代码）、反馈（测试通过/不通过）都是高度结构化的，天然适合 Agent 发挥。横轴看：编程 Agent 的用户（开发者）对 Agent 的容错率最高——代码报错能看 log、能回滚、能手动修复，不像订机票的 Agent 一旦订错就造成真实损失。编程 Agent 的率先商业化，不是偶然。

Agent "容错困境"是 2024–2026 年的核心矛盾。 2023 年的 Agent 最惊艳的地方是"它竟然能做！"，2024–2025 年的 Agent 最让人失望的地方是"它竟然会搞砸！"。用户的期望值在被 Demo 拉高后，真实体验跟不上的落差造成了所谓"Agent 冬天"的讨论。但我的判断是：这不是冬天，这是从"期望膨胀期"到"实质生产期"的必经阵痛。

5.2 竞品的纵向对比

如果把主要竞品也放到时间线上看：

- **LangChain（2022–12）** 是最早的 Agent 框架，靠"先发优势"积累了最大的生态。但它的旧架构（Chain 模型）已经被自己用 LangGraph 替代了。这有点像 Facebook 用 React Native 替代原生——自己革自己的命，是好事也说明当初的设计有问题。
- **AutoGPT（2023–03）** 是最早引爆 Agent 概念的项目，但后续发展乏力。作为个人项目（Significant Gravitas 一个人），它没能建立起可持续的开发团队和商业模式。AutoGPT 的星数（183k）和实际影响力之间的差距是所有 Agent 项目里最大的。
- **Dify（2023–05）** 在时间线上没有先发优势，但胜在定位清晰——不做框架做平台。Dify 的增长曲线是 2025–2026 年最陡的，说明当框架战争进入白热化时，通过"降维"（从代码到可视化）来竞争是有效的。
- **OpenAI Agents SDK（2025–03）** 是时间线上最新的框架。它的出现说明 OpenAI 认识到：光有模型和 API 不够，需要工具生态。但入场太晚，已经没法撼动 LangChain 和 Dify 的位置了。
- **OpenClaw（2025–11）** 是所有项目中"出道即巅峰"的典型——半年内从零到 GitHub 历史第一。它与其他框架的核心区别在于定位：不是"帮开发者写 Agent"的工具，而是"给最终用户用 Agent"的操作系统。OpenClaw 的爆发标志着 Agent 从"开发者工具"进入"消费者产品"阶段，这也是为什么它的 Star 数远超所有框架——它面向的不是开发者圈子，而是所有对 AI 感兴趣的人。
- **Hermes Agent（2026–03）** 是 OpenClaw 之后增长最快的 Agent 项目——两个月内 10 万 Stars。它的差异化打得很准：OpenClaw 没做好的"记忆"和"自我进化"是 Hermes Agent 的核心卖点。但它和 OpenClaw 走了同一条路——个人 Agent 操作系统——意味着它直接和 OpenClaw 抢用户，而不是创造新品类。这种"正面刚"的竞争策略短期内能借 OpenClaw 的知名度获益，但长期要看它能不能在"记忆"这个单一差异化上建立足够宽的护城河。

5.3 优势与劣势的历史根源

Agent 的核心优势——"自主性"——可以追溯到 ReAct 论文（2022）。 ReAct 的"推理+行动"循环是所有现代 Agent 的底层设计模式。所有的 Agent 框架、产品、基准测试，都是在优化这个循环的某个环节。

Agent 的核心劣势——"不可靠性"——则根植于 LLM 本身的随机性。 LLM 不是确定性系统，同样的输入在不同时间可能输出不同的结果。这对聊天场景没问题（甚至被看作"创造力"），但对 Agent 场景是致命的——一个需要精确执行的自动化系统，如果核心推理引擎是不确定的，可靠性天花板就是低的。这个问题不是工程优化能完全解决的，它可能需要回到模型层面（推理链路的可验证性、确定性的工具调用分支）。

"Demo 级 vs 生产级"的鸿沟，历史根源在 2023 年的 Agent 元年的成功标准。 2023 年的 Agent 项目比拼的是"能做什么"，而不是"做得多可靠"。AutoGPT 的 Demo 展示了一个自主运行的 Agent，但没有人去验证它在 100 次运行中

的成功率。这种"功能优先于可靠性"的惯性一直延续到 2025 年的 Manus。直到 2026 年的删库事件，行业才开始真正重视 Agent 的安全性和可靠性。

2026 年最深刻的商业模式变化：从"卖座位"到"卖工作量"。 Anthropic Enterprise 转向按用量计费、GitHub Copilot 转向 consumption pricing、Salesforce Agentforce 按对话收费——所有巨头都在同一时间抛弃了 SaaS 经典的 per-seat 订阅制。原因很直接：Agent 的工作量分布极度不均匀（一个月可能只用几次，也可能跑几千次），固定订阅下要么用户觉得亏要么厂商亏。按用量计费对双方都更公平，但也意味着 **Agent 的使用成本是透明且可量化的**——这对企业决策是好事，但对 Agent 厂商意味着"你的价值必须对得起每一次调用的价格"。

2026 年最大的格局变化：Anthropic 从模型公司变成了 Agent 平台公司。 从 Claude Agent SDK（2025.09）到 Cowork/Dispatch（2026.03）再到 Managed Agents（2026.04），Anthropic 在半年内完成了从"卖模型"到"卖 Agent 服务"的转型。这个决策的逻辑很清楚：Agent 消耗的 token 是普通聊天的 10-50 倍，做 Agent 产品比单纯卖 API 能 capture 更多价值。但对 Agent 创业公司来说，最大的模型供应商之一变成了直接的竞争对手。行业把这称为 **"平台挤压"**——当模型厂商自己做 Agent 时，依赖该模型的 Agent 创业公司的生存空间急剧缩小。

Anthropic 的安全路线选择可以追溯到 2023 年。 当 OpenAI 在 2023 年 6 月推出 Function Calling 时，Anthropic 的 Claude 对 Tool Use 采取了更保守的策略——模型在不确定时倾向不调用工具。2024 年的 Computer Use 也是先强调安全再展示能力。这种"安全优先"的路线让 Anthropic 在 Agent 的"快速迭代"阶段稍显落后，但在 2026 年安全事件频发后，变成了差异化优势。

5.4 未来推演：三个剧本

剧本一：最可能的剧本——Agent 成为"新型应用层"（概率 55%）

Agent 不会取代 App，也不会取代 OS，而是在现有基础设施上新增一层"智能中间层"。就像移动互联网没有取代 PC 互联网，而是创造了新的场景。Agent 的核心价值在于**连接**——连接不同的 SaaS 工具、连接线上和线下、连接数据和决策。

在这个剧本里：

- 框架格局两极化：OpenClaw 做消费者端个人 Agent，LangChain/Dify 做企业端 Agent 开发
- 编程 Agent 全面落地：Cursor 等成为开发标配
- 定价全面转向按用量计费，Agent 的使用成本透明化
- Agent 安全和治理成为成熟的企业级市场
- Anthropic 和 OpenAI 的 Agent 产品与第三方创业公司形成竞争

剧本二：最危险的剧本——"平台挤压 + 安全事故"双重打击（概率 25%）

两个风险叠加：Anthropic/OpenAI 亲自下场做 Agent 产品挤压了创业生态，导致创新活力下降；同时安全事故持续发酵，行业信心受挫。一个严重事故（比如 Agent 自动执行的金融交易导致亿元级损失）触发监管，Agent 应用受到严格限制。"Agent 冬天"真的到来。

在这个剧本里：

- Agent 创业公司大量倒闭（已出现苗头：Yupp.ai 倒闭、Robin AI 破产）
- 企业 Agent 部署大幅放缓
- 监管收紧，Agent 的自主权被严格限制

- 编程 Agent 受影响最小（代码领域风险可控）

这个剧本不是危言耸听——2026 年 4 月的删库事件如果发生在金融或医疗领域，后果要严重得多。行业在 Agent 安全上的投入还远远不够。"SaaSpocalypse"的讨论也在加剧传统软件厂商对 Agent 的警惕。

剧本三：最乐观的剧本——Agent 成为个人数字助理（概率 20%）

OpenClaw 的成功证明了"个人 Agent OS"的市场需求真实存在。Agent 不是取代 OS，而是成为运行在 OS 之上的"个人数字助理层"——用户通过 Agent 管理文件、操作应用、处理信息。MCP 成为无处不在的基础设施，类似 USB 之于外设。

在这个剧本里：

- Agent 平台的竞争进入白热化，OpenClaw 开源 vs 厂商闭源
- SaaS 公司从卖 App 变成卖 Agent API，UI 不再是核心产品
- "Agent-as-a-Service"的按用量定价成为标配
- 模型厂商同时做 Agent 产品成为常态，"平台挤压"加速行业洗牌

这个剧本比一年前更可信了——OpenClaw 的 30 万 Stars 和 Cowork/Dispatch 的发布都说明用户对"个人 Agent"的需求真实存在。但仍需跨越三个巨大障碍：

1. **可靠性**：Agent 需要达到 99.9%+ 的任务成功率，现在可能只有 80-90%
2. **成本**：Agent 的 token 消耗量是 chat 的数十倍，按用量计费下用户愿不愿意买单还是未知数
3. **安全**：Agent 权限控制、数据隐私、审计溯源都需要根本性的解决方案

如果把视角拉远，Agent 的发展实际上揭示了一个更大的趋势：**AI 的"行动化"**。从 2017 年 Transformer 让 AI "能理解"，到 2022 年 ChatGPT "能对话"，再到 2023 年 Agent "能行动"，AI 的能力边界在持续外扩。每一次外扩都伴随着控制权的重新分配——从"人类操作 AI"到"AI 自主行动"再到"人类监督 AI"。

这个趋势不会停止。2026 年的 Agent 可能还停留在"辅助工具"的阶段，但五年后呢？十年后呢？当 Agent 能够自主管理代码库、自主完成供应链调度、自主进行科学研究时，"控制权"这个问题的答案可能会让我们所有人都感到不安。

5.5 一条贯穿始终的暗线

回看整个 Agent 发展史，有一条暗线贯穿始终：**控制权的转移。**

SOAR 时代，控制权完全在程序员手里——你写什么规则，Agent 执行什么。2023 年，控制权转移到 LLM 手里——模型根据 prompt 自主决定做什么。2024-2025 年，控制权开始在"模型自主"和"人类审核"之间摇摆——Agent 自主行动，但关键步骤需要人工确认。

到了 2026 年，控制权的博弈进入了一个新阶段：**从"谁控制"变成了"如何可信地控制"。** Agent 可以自主行动，但必须有可审计的推理链、可回滚的操作日志、可拦截的安全护栏。

这场关于控制权的博弈，最终的决定因素可能不是技术能力，而是**信任**——人类多大程度上信任 Agent 替自己做决策。而信任，说到底是个经验问题，不是技术问题。就像自动驾驶需要累积足够多的安全行驶里程，Agent 也需要积累足够多的"无事故任务次数"，才能真正融入我们的日常生活。

七、实战：面试题、场景题、应用题

面试题

Q1: Tool Calling 和 Agent 的本质区别是什么？

考察对 Agent 概念的理解深度，而非复述定义。

核心区别：Tool Calling 是"模型能调用函数"，Agent 是"模型能自主决定何时、如何、按什么顺序调用函数，并在失败时调整策略"。Tool Calling 是一次性的——用户说"查一下北京天气"，模型调一个天气 API。Agent 是多步自主的——用户说"帮我安排去北京的旅行"，模型自己拆成查天气、比机票、找酒店、排行程，中间某个 API 失败会换策略而不是直接报错。

更深一层：Agent 的核心差异化不是"有工具"，而是**有循环**——ReAct 的"思考 → 行动 → 观察 → 思考"循环。这个循环让 Agent 从"能调 API 的聊天机器人"变成"能自主完成任务的数字员工"。

选型指南：

- 如果你的任务可以一步完成（查天气、翻译、总结）→ Tool Calling 就够了
- 如果你的任务需要多步推理和自主决策（订旅行、写代码、做竞品分析）→ 必须用 Agent

Q2: ReAct 为什么能工作？它解决的根本问题是什么？

考察对 Agent 核心机制的深层理解。

ReAct 解决的根本问题：LLM 是一个"闭门造车"的系统——只能基于训练数据生成回答，无法感知和验证外部世界。你问"今天北京飞上海的航班有哪些"，纯 LLM 只能编造航班号。ReAct 的核心洞察是：**把推理（Reasoning）和行动（Acting）交替进行**，让模型每推理一步就先去外部世界验证一下，然后基于真实信息再推理下一步。

这个机制之所以有效，是因为它把 LLM 从"一次性生成答案"变成"迭代式探索和验证"。说到底，ReAct 在模拟人类解决复杂问题的方式——先想一步、做一点、看看结果、再想下一步。

局限：ReAct 在多步任务上可能陷入"循环但无进展"的状态——模型反复推理和查询但始终没有接近答案。这引出了 Reflexion（反思，让模型从失败中学习）和 Tree-of-Thought（多路径探索，多条推理链并行）等改进方向。

Q3: MCP 和 A2A 各解决什么问题？为什么需要两个协议？

考察对 Agent 基础设施协议的理解。

MCP（Model Context Protocol）解决"Agent 怎么连接工具"——Agent 和单个工具之间的通信协议，类似 USB-C 之于外设。MCP 标准化了 Agent 如何发现工具、调用工具、接收返回结果，让不同 Agent 可以用统一方式连接不同工具。

A2A（Agent-to-Agent Protocol）解决"Agent 之间怎么通信"——当多 Agent 需要协作完成复杂任务时，A2A 定义了 Agent 之间交换信息、委托任务、汇报进度的标准方式。

为什么需要两个？它们解决不同层次的通信：MCP 是 Agent ↔ 工具的垂直通信，A2A 是 Agent ↔ Agent 的水平通信。一个 Agent 可能同时用 MCP 调用浏览器和代码执行器，同时用 A2A 与另一个 Agent 协调任务分工。**MCP 是"Agent 的手"，A2A 是"Agent 之间的语言"。**

场景题

场景 1：为电商客服设计 Agent 的安全边界

> 你要为电商平台设计一个 AI 客服 Agent，可以访问订单数据库、物流 API、商品目录。用户会问订单状态、退换货流程、商品推荐等。Agent 的权限边界和安全策略怎么设？

第一步：定义工具权限。Agent 需要三个只读工具：`query_order(order_id)`、`query_logistics(tracking_number)`、`search_products(query)`。核心原则——Agent 只能**查询**，绝不能**修改**订单状态或发起退款。写入操作必须由人类客服确认后执行。

第二步：安全边界设计。从四个层次控制：

- **Prompt 层**：System prompt 明确列出禁止操作（修改订单、发起付款、删除数据、访问用户敏感信息）
- **工具层**：工具注册时只开放查询类 API，删除/写入/退款接口根本不暴露给 Agent
- **输入层**：对用户输入做 prompt injection 检测——拒绝"忽略之前所有指令"类输入
- **业务规则层**：硬编码规则兜底——单笔退款超过 ¥500 必须人工、批量操作一律拦截

第三步：降级策略。定义 Agent 需要转人工的场景：用户表达愤怒、涉及金钱纠纷、Agent 连续两轮无法理解用户意图、任务超出 Agent 能力范围。转人工时附带 Agent 已收集到的信息，避免用户重复描述问题。

场景 2：Agent 失控——排查与防护

> 你团队的 Agent 在测试中出现了意外行为：它本应帮用户做竞品分析，但调用浏览器时访问了不该访问的页面，且试图调用文件系统的删除操作。你的排查路径和长期防护方案是什么？

排查路径（从急到缓）：

1. **先止损**：立即断开 Agent 的网络和文件系统权限，保留所有日志用于分析
2. **复现**：是用户恶意输入（prompt injection）还是模型自己"越界"？检查 Agent 的完整推理链日志——thought-action-observation 循环的每一步
3. **定位根因**：最常见三个原因——① 工具权限过大（Agent 的浏览器能访问整个互联网，文件系统工具有写/删权限）；② System prompt 未设边界；③ Prompt injection 注入成功

长期防护方案（分层）：

层次	措施	说明
Prompt 层	System prompt 明确禁止删除、写入、访问敏感 URL	第一道防线，但不可单独依赖
工具层	最小权限原则——浏览器只允许 GET，文件系统只读	Agent 根本拿不到危险操作的"手柄"
沙箱层	Agent 运行在 Docker 隔离容器中，所有操作在沙箱内执行	即使失控，破坏也只限于沙箱内
监控层	实时监控工具调用日志，异常行为（调用删除 API、访问内网 IP）立即告警并阻断	尽早发现、尽早拦截
人工审核层	高风险操作（支付、数据修改）必须人类确认	最后一道防线

核心原则：永远不要信任 Agent 的判断，用多层防护代替单层信任。Agent 能做什么由系统约束定义，不由 Agent 自己判断。

应用题

应用 1：设计一个代码审查多 Agent 系统

> 你所在团队每天有 20–30 个 PR 需要审查。设计一个多 Agent 系统，让 AI 自动完成第一轮代码审查，人类审查者只做最终确认。给出 Agent 分工、协作流程、质量标准、以及如何防止 Agent 漏掉严重 bug。

Agent 分工（三个专门 Agent，各自专注一个维度）：

Agent A — 安全审查：只找安全问题。检查 SQL 注入（字符串拼接）、XSS（未转义的用户输入）、敏感信息硬编码（API key、密码）、权限绕过、依赖中已知 CVE。安全规则尽量用确定性的模式匹配（Semgrep/CodeQL），Agent 做补充判断。输出：安全问题列表（按严重程度排序），无问题则输出"PASS"。

Agent B — 逻辑正确性：只找逻辑 bug。检查空指针/None、数组越界、并发问题（竞态条件）、事务边界、错误处理是否完备。输出：逻辑问题列表，每个问题标注置信度。

Agent C — 代码质量：只找可维护性问题。检查命名是否清晰、函数是否过长、是否违反单一职责、是否有重复代码、关键路径测试覆盖。输出：质量建议列表（全部为建议，不阻断合并）。

协作流程：

```
PR 创建
  ↓
三个 Agent 并行审查（各自独立，互不干扰）
  ↓
汇总 Agent 合并三个 Agent 的输出，去重，按严重程度排序
  ↓
汇总报告发给人类审查者
  ↓
人类确认 AI 标记的严重问题 + 补充遗漏 → 做最终 approve/reject
```

为什么用多 Agent 而非单 Agent：单 Agent 同时关注安全、逻辑、风格三个维度时容易"注意力稀释"——安全问题和代码风格问题被放在同一个 prompt 里，模型倾向于在两者之间平均分配注意力。多 Agent 各自专注一个维度，类同代码审查中"安全工程师查安全、模块 owner 查逻辑、Tech Lead 查质量"的分工逻辑。

防漏报策略：

- 安全 Agent 的漏报不可接受——所有安全规则必须有确定性检查兜底，Agent 只在规则引擎结果上做补充
- 每个发现标注置信度，低置信度标记"需要人类确认"
- 每周抽查 10 个 PR，对比 Agent 和人类审查结果，计算漏报率和误报率，根据数据调整 prompt 和规则
- **最关键的决策**：AI Agent 做第一轮审查但永远不做最终决定。最终 approve 权力在人类手里。这不是因为 AI 不够好——而是 Agent 漏掉严重 bug（假阴性）的代价极高，且人类审查者需要至少看过 PR 才能对代码有 ownership。

八、信息来源

核心论文与文章

论文/文章	来源	访问时间
ReAct: Synergizing Reasoning and Acting in Language Models	arXiv:2210.03629 (ICLR 2023)	2026-04-30
Chain-of-Thought Prompting Elicits Reasoning in LLMs	arXiv:2201.11903 (2022)	2026-04-30
Toolformer: LMs Can Teach Themselves to Use Tools	arXiv:2302.04761 (Meta, 2023)	2026-04-30
Reflexion: Language Agents with Verbal Reinforcement Learning	arXiv:2303.11366 (2023)	2026-04-30
Generative Agents: Interactive Simulacra of Human Behavior	arXiv:2304.03442 (Stanford, 2023)	2026-04-30
Tree of Thoughts: Deliberate Problem Solving with LLMs	arXiv:2305.10601 (NeurIPS 2023)	2026-04-30
Voyager: Open-Ended Embodied Agent with LLMs	arXiv:2305.16291 (NVIDIA, 2023)	2026-04-30
Graph of Thoughts: Solving Elaborate Problems with LLMs	arXiv:2308.09687 (AAAI 2024)	2026-04-30
SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering	arXiv:2405.15793 (2024)	2026-04-30
SWE-Edit (Microsoft)	arXiv:2604.26102 (2026)	2026-04-30
Recursive Multi-Agent Systems	arXiv:2604.25917 (2026)	2026-04-30
OxyGent: Observable Multi-Agent Systems	arXiv:2604.25602 (2026)	2026-04-30
Pythia: Agent-Native LLM Serving	arXiv:2604.25899 (2026)	2026-04-30
Conditional Misalignment	arXiv:2604.25891 (2026)	2026-04-30
SOAR 架构	卡内基梅隆大学，1983	2026-04-30
《Society of Mind》	Marvin Minsky，1986	2026-04-30

BDI 模型	Bratman, Rao & Georgeff，1987–1995	2026–04–30
"AI: A Modern Approach"	Russell & Norvig，1995	2026–04–30

产品与技术来源

来源	URL	访问时间
OpenAI Function Calling 发布	https://openai.com/blog/function-calling-and-other-api-updates	2026-04-30
Anthropic Computer Use 发布	https://www.anthropic.com/news/3-5-models-and-computer-use	2026-04-30
MCP (Model Context Protocol) 发布	https://www.anthropic.com/news/model-context-protocol	2026-04-30
MCP 捐赠给 Linux 基金会	https://www.anthropic.com/news/donating-the-model-context-protocol	2026-04-30
A2A (Agent-to-Agent Protocol)	https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/	2026-04-30
Devin 发布	https://www.cognition.ai/blog/introducing-devin	2026-04-30
OpenAI Operator / Agents SDK	openai.com (2025-01 / 2025-03)	2026-04-30
OpenAI Agents SDK v0.14.0 升级（Sandbox + Harness）	https://openai.com/sl-SI/index/the-next-evolution-of-the-agents-sdk/	2026-04-30
OpenAI Workspace Agents	openai.com (2026-04)	2026-04-30
Anthropic Claude Cowork + Dispatch	anthropic.com (2026-03)	2026-04-30
Anthropic Claude Managed Agents + Agent Skills	anthropic.com (2026-04)	2026-04-30
Google Cloud Next '26 Enterprise Agent Platform	cloud.google.com (2026-04)	2026-04-30
Microsoft Agent Framework 1.0	microsoft.com (2026-04)	2026-04-30
OpenClaw GitHub (300k+ Stars)	https://github.com/openclaw/openclaw	2026-04-30
Anthropic 可能融资 $500 亿	TechCrunch (2026-04-29)	2026-04-30
Parallel Web Systems 估值 $20 亿	TechCrunch (2026-04-29)	2026-04-30

中国否决 Meta 收购 Manus	TechCrunch (2026-04-27)	2026-04-30
OpenAI 正开发 Agent 手机	TechCrunch (2026-04-27)	2026-04-30
AWS 推出 OpenAI Agent 服务	TechCrunch (2026-04-28)	2026-04-30
Red Hat OpenClaw 容器化 Agent	TechCrunch (2026-04-28)	2026-04-30
AI Agent 删库事件	Hacker News (2026-04-28)	2026-04-30
MIT Tech Review: 炒作与盈利之间缺失的一环	technologyreview.com (2026-04-27)	2026-04-30
GitHub Copilot 转向用量计费	Hacker News (2026-04-29)	2026-04-30
Cursor Camp	Hacker News (2026-04-29)	2026-04-30
Claude Code 代码版权讨论	Hacker News (2026-04-29)	2026-04-30
GitHub Stars 数据	GitHub API (2026-04-30)	2026-04-30
SWE-Bench Leaderboard	swebench.com	2026-04-30
GAIA 基准测试	Meta FAIR 官方报告	2026-04-30
框架仓库参考	LangChain、Dify、AutoGen、CrewAI、MetaGPT 等开源项目	2026-04-30

> **方法论说明**：本报告采用横纵分析法（Horizontal-Vertical Analysis），由数字生命卡兹克（Khazix）提出，融合了语言学中的历时-共时分析（Saussure）、社会科学中的纵向-横截面研究设计、商学院案例研究法、以及竞争战略分析的核心思想。纵向维度追踪 AI Agent 从 1983 年至今的完整发展史，横向维度在 2026 年 4 月的时间切面上对比框架、产品、大厂战略与基准表现，最终交汇产出对 Agent 未来走向的判断。

本文是横纵分析系列的第 8 篇报告。方法论：横纵分析法——纵向沿时间轴追溯技术演进，横向对比当下竞争格局。